# Heterogeneous treatment effects

**Lecture notes**

Foundations of Regression and Predictive Modeling

## Table of contents


From "does it work?" to "for whom?" . . . . . 1
Case 1: the win-back offer . . . . . 2
Treatment-by-group interactions . . . . . 2
A numeric moderator . . . . . 4
Subgroup fishing . . . . . 5
Case 2: dose-response . . . . . 7
The effect curve and a targeting rule . . . . . 8
Why heterogeneity is worth the trouble . . . . . 10
Summary . . . . . 10


### From "does it work?" to "for whom?"

Every experiment in this unit so far has ended with one number: the treatment effect. That number is an average over everyone in the experiment, and averages can conceal as much as they reveal. A treatment that does nothing on average may work well for one group and backfire for another; a treatment with a healthy average effect may earn all of it from a minority of units while wasting money on the rest.

This module is about modeling that variation. The tools are not new — they are the interaction terms from the categorical-variables and factorial-experiment modules, plus the spline machinery from the smoothing module — but the target changes. Instead of modeling how the *outcome* varies, we model how the *treatment effect* varies: across groups, along a numeric covariate, and along the dose of the treatment itself.

One warning runs through everything here. Effect differences are estimated with less data than effects: each subgroup has a fraction of the sample, and a difference of two noisy numbers is noisier still. Heterogeneity claims therefore deserve more skepticism than average-effect claims, and subgroups invented after peeking at the data deserve the most skepticism of all.

## Case 1: the win-back offer

The Austin pet-supply retailer from the factorial-experiment module is still running tests. The growth team emails a win-back offer — 30% off your next order — to a random half of 4,000 customers who have not ordered recently. The outcome is spend over the following 60 days.

```
winback <- read_csv(file.path(module_dir, "winback_offer.csv")) |>
  mutate(segment = factor(segment, levels = c("active", "drifting", "lapsed")))

winback |> head(4)
```

```
# A tibble: 4 x 7
  customer_id weeks_since_last_order segment offer app_platform signup_weekday
  <chr>                        <dbl> <fct>   <dbl> <chr>        <chr>
1 W61177                        26.9 lapsed      0 android      Wed
2 W34176                         2   active      1 ios          Fri
3 W95564                         2.7 active      1 ios          Tue
4 W33769                         3.9 active      1 android      Sat
# i 1 more variable: spend_60d <dbl>
```

The `segment` column comes from the CRM's lifecycle labels, defined before the experiment ran: **active** customers ordered within the last 4 weeks, **drifting** customers have been quiet for 4 to 12 weeks, and **lapsed** customers for more than 12.

The average treatment effect is the one-dummy regression from the start of this unit:

```
overall_model <- lm(spend_60d ~ offer, data = winback)
tidy(overall_model, conf.int = TRUE)
```

```
# A tibble: 2 x 7
  term        estimate std.error statistic     p.value conf.low conf.high
  <chr>          <dbl>     <dbl>     <dbl>       <dbl>    <dbl>     <dbl>
1 (Intercept)    46.8      0.545     85.9  0              45.8      47.9
2 offer           4.13     0.775      5.33 0.000000105     2.61      5.65
```

About +$4.13 per customer, clearly positive. Since the email costs almost nothing, the tempting conclusion is a blanket rollout. But the $4.13 averages over a customer who ordered last week, a customer quietly drifting away, and a customer gone for six months, and there is no reason one discount should do the same thing to all three.

## Treatment-by-group interactions

The interaction model lets each segment have its own treatment effect:

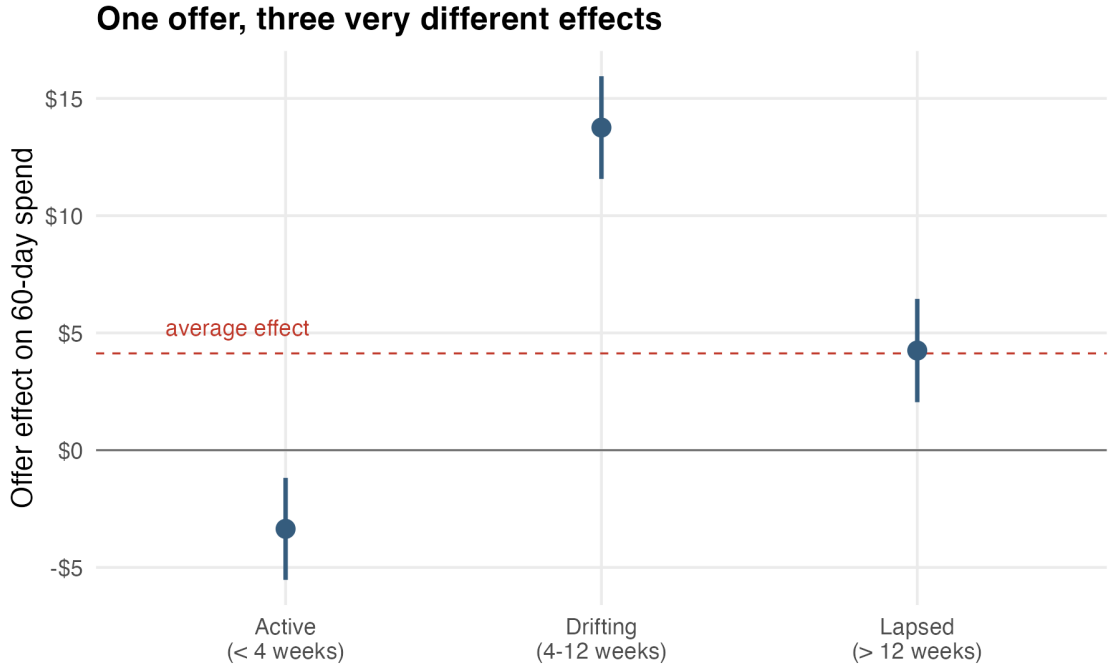
```r
interaction_model <- lm(spend_60d ~ offer * segment, data = winback)
tidy(interaction_model, conf.int = TRUE)
```

```
# A tibble: 6 x 7
  term                 estimate std.error statistic   p.value conf.low conf.high
  <chr>                   <dbl>     <dbl>     <dbl>     <dbl>    <dbl>     <dbl>
1 (Intercept)             61.6      0.760     81.0  0            60.1      63.1
2 offer                   -3.35     1.09      -3.08 2.08e-  3    -5.49     -1.22
3 segmentdrifting        -13.7      1.07     -12.7  2.66e- 36   -15.8     -11.5
4 segmentlapsed          -34.5      1.14     -30.3  1.14e-181   -36.8     -32.3
5 offer:segmentdrifti~    17.1      1.54      11.1  3.41e- 28    14.1      20.1
6 offer:segmentlapsed      7.60     1.61       4.74 2.24e-  6     4.46     10.8
```

The reading follows the factorial module, with one twist in emphasis:

- `offer` is the treatment effect *in the baseline segment* (active): about −$3.35.
- Each interaction coefficient is the *difference between that segment's effect and the baseline effect*. The drifting effect is $-3.35 + 17.11 \approx +\$13.76$; the lapsed effect is $-3.35 + 7.60 \approx +\$4.25$.

The interaction coefficients come with their own intervals — `offer:segmentdrifting` is +17.11 with interval roughly $[14.09, 20.14]$ — so "the segments differ" is itself an estimated, uncertainty-quantified claim, not an eyeball judgment.




The active-segment result deserves attention: the offer *reduced* revenue among customers who ordered recently, with an interval entirely below zero. Nothing mysterious happened — these customers were about to order anyway, and the offer took 30% off purchases that would have

occurred at full price. That is **cannibalization**, and only a heterogeneity analysis can see it. The average effect quietly nets the drifting segment's gains against the active segment's losses.

The decision changes accordingly. A blanket rollout earns the average; a targeted rollout — send to drifting and lapsed, exclude active — earns the average *plus* the money the active segment was losing. Same experiment, same data, roughly double the payoff.

### A numeric moderator

The segments are bins of a number the retailer already has: weeks since the last order. The direct way to use the number is a treatment-by-covariate interaction:

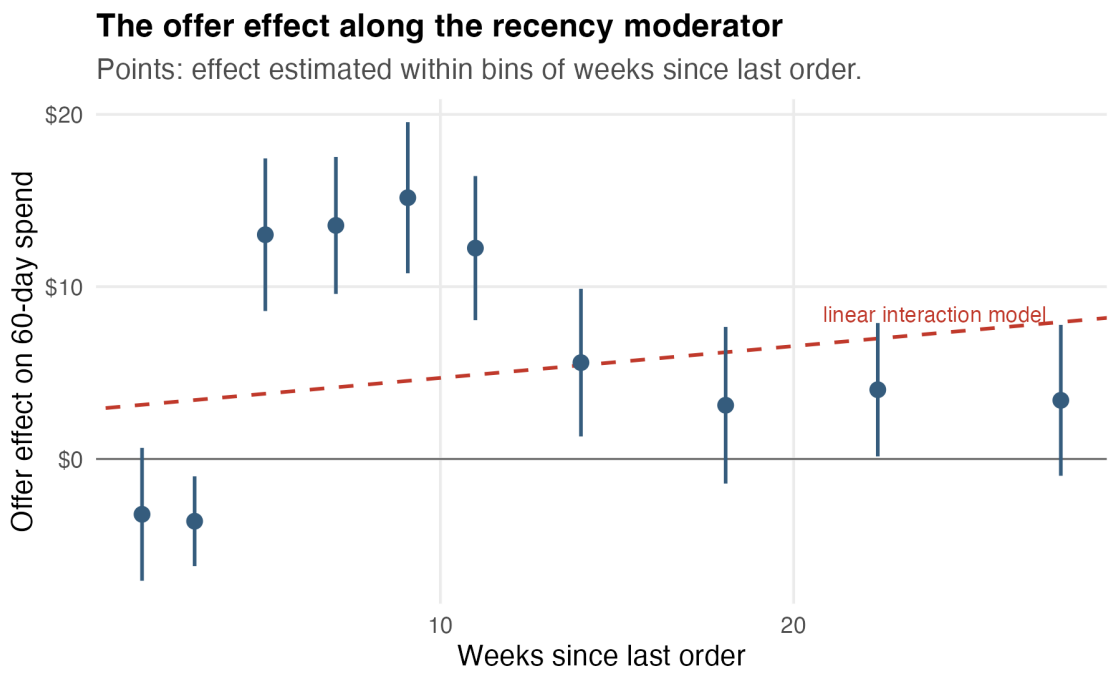
```
weeks_model <- lm(spend_60d ~ offer * weeks_since_last_order, data = winback)
tidy(weeks_model, conf.int = TRUE)
```

```
# A tibble: 4 x 7
  term                 estimate std.error statistic   p.value conf.low conf.high
  <chr>                   <dbl>     <dbl>     <dbl>     <dbl>    <dbl>     <dbl>
1 (Intercept)             64.0      0.713      89.8  0          62.7       65.4
2 offer                    2.86     1.02        2.81 5.01e-  3   0.863      4.85
3 weeks_since_last_or~    -1.77     0.0565    -31.3  1.10e-192  -1.88      -1.66
4 offer:weeks_since_l~     0.185    0.0795      2.32 2.03e-  2   0.0287     0.340
```

The interaction says the offer effect grows by about \$0.18 per week of recency, with an interval of roughly $[0.03, 0.34]$. Statistically significant, tidy — and wrong about the shape. Estimating the effect within recency bins tells a different story:

```
winback |>
  mutate(bin = cut(weeks_since_last_order, c(0, 4, 8, 12, 20, 30))) |>
  group_by(bin) |>
  group_modify(\(d, g) {
    tidy(lm(spend_60d ~ offer, data = d), conf.int = TRUE) |>
      filter(term == "offer")
  }) |>
  select(bin, estimate, conf.low, conf.high)
```

```
# A tibble: 5 x 4
# Groups:   bin [5]
  bin     estimate conf.low conf.high
  <fct>      <dbl>    <dbl>     <dbl>
1 (0,4]      -3.35   -5.53      -1.18
2 (4,8]      13.2    10.2       16.1
3 (8,12]     13.9    10.9       17.0
4 (12,20]     3.79    0.646      6.93
5 (20,30]     3.67    0.782      6.56
```




The real effect rises and then falls: negative for recent customers, peaked in the drifting window, fading for the long-lapsed. A single linear interaction cannot bend, so it reported the average tilt of that shape and missed both the peak and the negative region. The smoothing module's lesson applies to effects, not just outcomes: numeric moderators may need curves.

### Subgroup fishing

The CRM has dozens more columns, and ten more subgroup analyses cost ten lines of code. The dataset includes two columns — app platform and signup weekday — that were generated with *no connection whatsoever* to the offer. Testing an offer interaction for ten junk splits produces this:

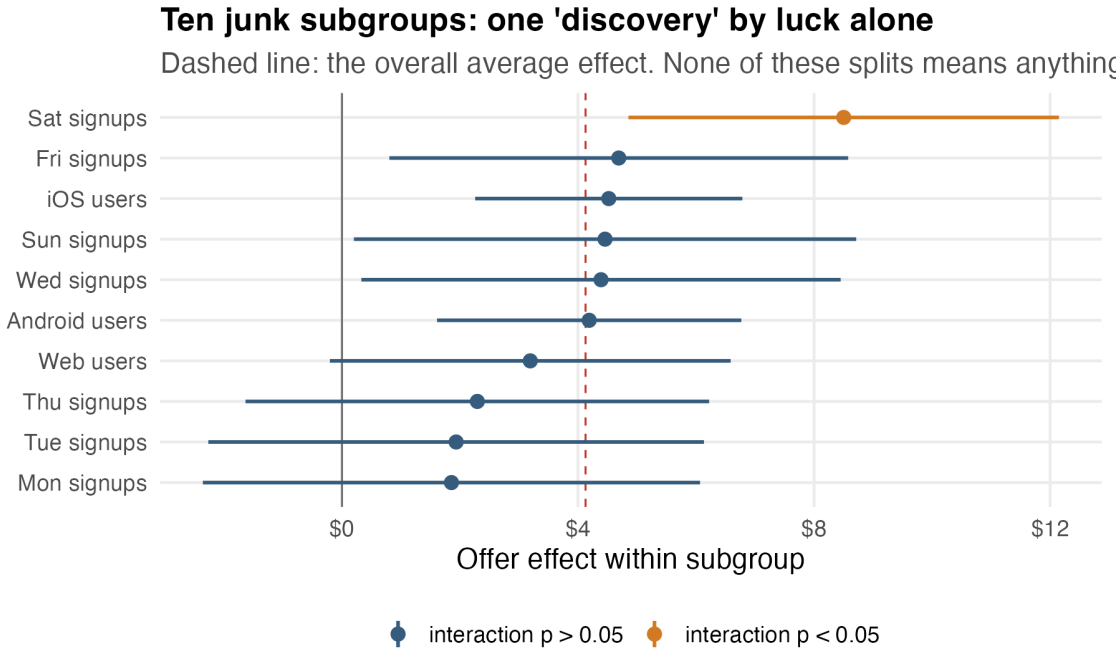
```
junk_splits <- list(
  ios = winback$app_platform == "ios",
  android = winback$app_platform == "android",
  web = winback$app_platform == "web",
  Mon = winback$signup_weekday == "Mon",
  Tue = winback$signup_weekday == "Tue",
  Wed = winback$signup_weekday == "Wed",
  Thu = winback$signup_weekday == "Thu",
  Fri = winback$signup_weekday == "Fri",
  Sat = winback$signup_weekday == "Sat",
  Sun = winback$signup_weekday == "Sun"
)

imap_dfr(junk_splits, \(flag, label) {
  d <- winback |> mutate(in_group = as.integer(flag))
  lm(spend_60d ~ offer * in_group, data = d) |>
```

```
    tidy() |>
    filter(term == "offer:in_group") |>
    transmute(subgroup = label, interaction_p = p.value)
}) |>
  arrange(interaction_p)
```

```
# A tibble: 10 x 2
   subgroup interaction_p
   <chr>            <dbl>
 1 Sat             0.0163
 2 Tue             0.242
 3 Mon             0.245
 4 Thu             0.340
 5 web             0.542
 6 ios             0.647
 7 Fri             0.757
 8 Sun             0.861
 9 Wed             0.886
10 android         0.959
```

Saturday signups show an offer effect of about +$8.50 — double the average — with an interaction p-value of 0.016. It is pure luck. With ten tests at the 5% level, the chance of at least one false hit is $1 - 0.95^{10} \approx 40\%$, and subgroup estimates are noisy enough (smaller $n$, wider intervals) to swing dramatically by chance.




The recency segments and the Saturday finding have different pedigrees. The segments were **pre-specified**: named before launch, with a business reason attached. Saturday was **discovered**:

found by searching after the fact, with an explanation invented afterward. The rules of engagement for discoveries: demote them to hypotheses, replicate them in the next experiment before acting, and if searching is unavoidable, count the searches and correct for them.

### Case 2: dose-response

The retailer's second question is not whether a treatment works but *how much treatment to give*. The media buyer wants to know how many retargeting impressions per week to show one person. The experiment randomizes 3,600 customers to a weekly impression cap of 0, 2, 4, 8, 12, or 16, with spend over 30 days as the outcome.

```
adcap <- read_csv(file.path(module_dir, "ad_frequency.csv"))

adcap |>
  group_by(weekly_ad_cap) |>
  summarize(n = n(), mean_spend = mean(spend_30d))
```

```
# A tibble: 6 x 3
  weekly_ad_cap     n mean_spend
          <dbl> <int>      <dbl>
1             0   600       40.1
2             2   600       44.0
3             4   600       49.7
4             8   600       50.8
5            12   600       51.4
6            16   600       51.4
```

Spend climbs steeply from cap 0 to cap 8, then stops climbing. The straight-line dose model misses that structure:

```
linear_dose <- lm(spend_30d ~ weekly_ad_cap, data = adcap)
tidy(linear_dose, conf.int = TRUE)
```

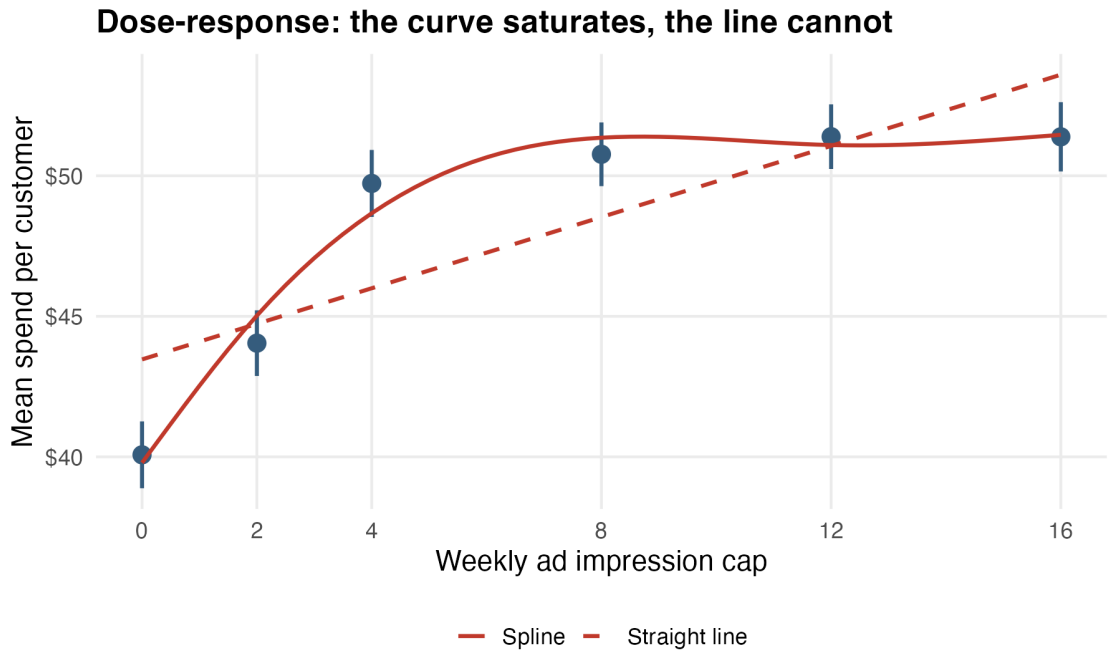
```
# A tibble: 2 x 7
  term          estimate std.error statistic  p.value conf.low conf.high
  <chr>            <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
1 (Intercept)     43.5      0.397      110.  0          42.7      44.2
2 weekly_ad_cap    0.633    0.0442      14.3 2.25e-45    0.547     0.720
```

The line promises about 63 cents per impression, forever — extrapolate to 30 impressions and it forecasts +$19 per customer, which no advertiser has ever experienced. The response saturates, so the model must be allowed to bend. This is the spline machinery from the smoothing module, applied to a treatment dose:

```r
spline_dose <- lm(spend_30d ~ ns(weekly_ad_cap, df = 4), data = adcap)

tibble(weekly_ad_cap = c(0, 4, 8, 16)) |>
  mutate(fitted_spend = predict(spline_dose, newdata = pick(everything()))) |>
  mutate(gain_from_previous = fitted_spend - lag(fitted_spend))
```

```
# A tibble: 4 x 3
  weekly_ad_cap fitted_spend gain_from_previous
          <dbl>        <dbl>              <dbl>
1             0         39.8             NA
2             4         48.7              8.88
3             8         51.4              2.69
4            16         51.5              0.103
```




The fitted curve turns the experiment into a marginal-value table: the first four impressions earn about $2.20 each, the next four about 67 cents each, and impressions beyond eight buy about a penny apiece while costing the same to serve. The decision is a cap at 8. What answered the question was not a treatment effect but a *marginal* treatment effect read off a fitted curve — dose-response is heterogeneity along the treatment itself.

### The effect curve and a targeting rule

The win-back case left two loose ends. The binned effects traced a rise-and-fall shape that no linear interaction could follow, and the three segments are crude bins of a number the retailer knows exactly for every customer. The dose-response case supplied the missing tool — splines — so the

closing move is to put the spline where the heterogeneity is and let the treatment effect itself be a smooth curve in recency:

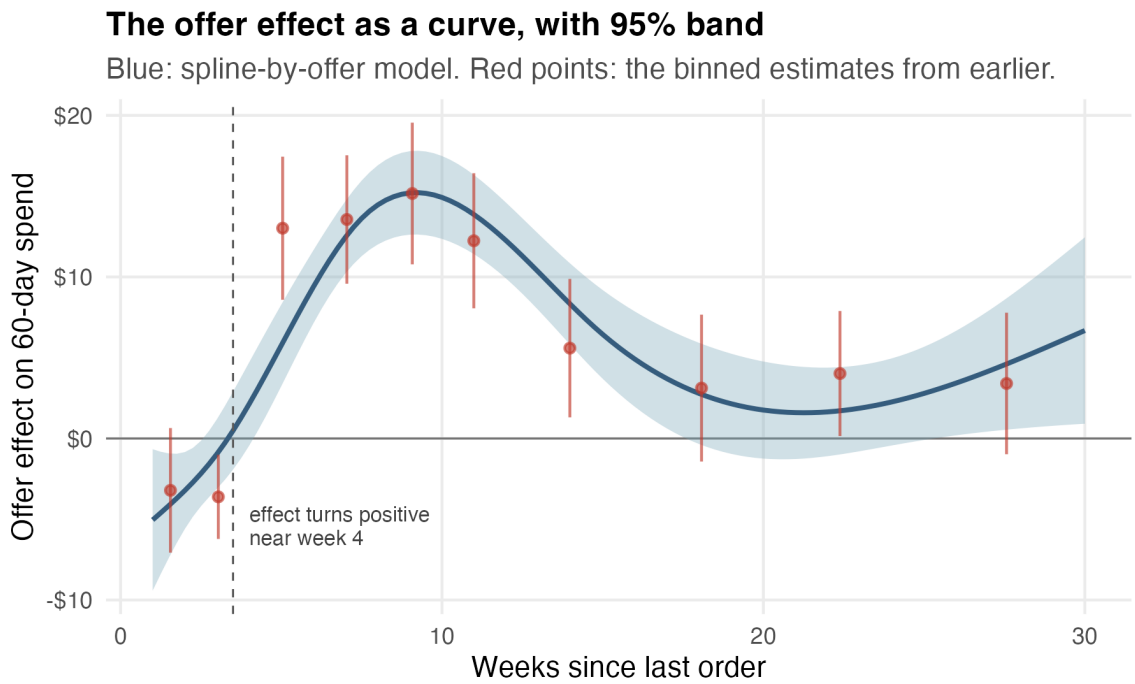
```
curve_fit <- lm(spend_60d ~ ns(weeks_since_last_order, df = 4) * offer, data = winback)
```

The model fits two smooth curves, one per arm, and the treatment effect at recency $w$ is the vertical gap between them, $\hat{\tau}(w) = \hat{f}_1(w) - \hat{f}_0(w)$. Each gap is a linear combination of coefficients — a contrast, exactly as in the experiments module — so it carries a standard error from the coefficient covariance matrix:

```
curve_grid <- seq(1, 30, by = 0.25)
nd1 <- tibble(weeks_since_last_order = curve_grid, offer = 1)
nd0 <- tibble(weeks_since_last_order = curve_grid, offer = 0)

tt <- delete.response(terms(curve_fit))
D <- model.matrix(tt, nd1) - model.matrix(tt, nd0)

effect_curve <- tibble(
  weeks = curve_grid,
  effect = drop(D %*% coef(curve_fit)),
  se = sqrt(rowSums((D %*% vcov(curve_fit)) * D))
)
```




The curve says everything the segments said, plus where the boundaries actually are: the offer loses money below about week 4, peaks near +$15 around week 9, and fades to a few dollars for the long-lapsed, with a band that widens where the data thin out. The right-hand edge wiggles —

splines always do at boundaries — which is a reminder to trust the curve where the observations are dense.

A curve supports a decision per customer rather than per bin: send the offer wherever $\hat{\tau}(w) > 0$, which here means customers quiet for more than about four weeks — roughly 73% of the list. Scoring every customer with the model prices the policies for a 100,000-customer file: the blanket policy is worth about \$478K of incremental spend, and the curve-targeted policy about \$552K while sending 27% fewer emails. The entire gain is the active segment's negative effect, turned off.

Two caveats belong in the write-up rather than the footnotes. First, the rule was built and priced on the same experiment; a cutoff chosen where a fitted curve crosses zero inherits the curve's noise, and same-data payoff estimates lean optimistic. The honest procedure is to freeze the rule and validate it on the next campaign. Second, the curve exists only where the data do — nobody in this experiment had been quiet for 40 weeks, so the rule says nothing about such customers. Within those limits, an effect curve estimated from a randomized experiment is the honest version of personalization: every promise it makes is a contrast between randomized groups.

## Why heterogeneity is worth the trouble

Heterogeneity analysis is where experiments meet decisions:

- **Targeting.** Send the offer where it earns; turn it off where it cannibalizes.
- **Rollout.** Launch where the effect is demonstrated; keep testing where it is not.
- **Dosing.** Spend up to the flat part of the curve, then stop.
- **Fairness.** "Works on average" can coexist with "fails for an important group." When treatment effects differ across groups of people, the average hides who pays for the win, and subgroup reporting is an honesty requirement, not a statistical nicety.

An average effect is a summary. A heterogeneity analysis is a plan.

## Summary

- The average treatment effect answers "does it work on net?" Rollout decisions usually need "for whom?", and the two answers can differ in sign, as the active segment showed.
- Treatment-by-group interactions estimate differences between treatment effects, with intervals of their own. Read the treatment coefficient as the baseline-group effect and each interaction as an offset to it.
- Treatment effects can vary along a numeric covariate, and a linear interaction reports only the tilt. Check the shape with binned effects, and let moderators have curves when the data ask for them.
- Searching many subgroups guarantees occasional false discoveries — about a 40% chance in ten looks. Prespecify subgroups where possible; treat discovered ones as hypotheses to replicate.
- Dose-response models the effect as a curve in treatment intensity. Fit it with splines, read marginal gains off the curve, and stop paying past the saturation point.
- This closes the experiments unit: designed comparisons (randomization), nuisance control (blocks and fixed effects), found comparisons (before/after and thresholds), and effects that vary (heterogeneity).